アルゴリズム特論｜第８回

型や型クラスを自分で作ろう

7.6–7.11

前川研究室
中谷尚史

7.6 型シノニムとは

型シノニムは、既存の型に別名を付ける仕組み。

```
type String = [Char]
```

Stringと[Char]は同じ型。
Charは「1文字」、[Char]は「文字のリスト」。

typeは新しいデータ型や値コンストラクタを作らない。
新しいデータ型を定義するのはdata。

別名を付けることで、
型宣言からデータの役割を読み取りやすくする。

「シノニム」の由来

synonymは英語で「同義語・類義語」。
語源はギリシャ語で、「名前」を表すonymaが含まれる。
型シノニムも、同じ型に別の名前を付ける仕組み。

メリット：型宣言の違い

同じ関数の型を、二通りに書ける。

```
toUpperString :: [Char] -> [Char]
```

Stringという別名を使うと、文字列を扱うことが読み取りやすい。

```
toUpperString :: String -> String
```

二つは同じ型を表す。
関数の処理や実行結果を変更する仕組みではない。

注意：NameとPhoneNumberを両方Stringの別名にしても、
取り違えを型で防げるわけではない。

課題1 電話帳をかっこよくしよう

課題

次の型宣言を、役割が伝わる型シノニムを使って書き換えよう。

```
inPhoneBook :: String -> String
            -> [(String, String)] -> Bool
```

NameとPhoneNumberをStringの別名として定義する。
名前と電話番号の組のリストに、PhoneBookという別名を付ける。

関数の本体は変更せず、型宣言だけを書き換える。

課題1 電話帳をかっこよくしよう

穴埋めヒント

```
type Name = __A__
type PhoneNumber = __A__
type PhoneBook = [(__B__, __C__)]
```

```
inPhoneBook :: __B__ -> __C__
            -> PhoneBook -> Bool
```

A〜Cに型名を入れよう。
同じ記号の空欄には、同じ型名が入る。

課題1 電話帳をかっこよくしよう

解答例

```
type Name = String
type PhoneNumber = String
type PhoneBook = [(Name, PhoneNumber)]
```

```
inPhoneBook :: Name -> PhoneNumber
            -> PhoneBook -> Bool
```

型の役割が読み取りやすくなる。
いずれも元の型の別名なので、処理や実行結果は変わらない。

7.6型シノニムの多相化

要点：

- 型シノニムは型引数を取ることができる
- 一般的なデータ構造を定義して、様々な型に使いまわせる

AssocList(アソシエーションリスト・連想リスト)

Int Map（整数キーマップ）

7.6型シノニムの多相化

AssocList(アソシエーションリスト・連想リスト)

次のような定義を見てください

```
type AssocList k v = [(k, v)]
```

ここではkとvは型変数です。

つまりAssocListは「キーがk型」「値がv型」のペアリストを表している

7.6 型シノニムの多相化：使用例

同じ連想リストを、キーと値の型を変えて使える。

```
type Name = String
type PhoneNumber = String
type PhoneBook = AssocList Name PhoneNumber
```

PhoneBookは、名前と電話番号の組のリストに付けた別名。

```
type GradeBook = AssocList Int Char
type PriceTable = AssocList String Int
```

```
AssocList String Int
-- [(String, Int)]と同じ型
```

型引数を渡すことで用途に応じた型を表せる。
型シノニムの名前で値を作るわけではない。

7.6 型シノニムの多相化：IntMap


```
import qualified Data.Map as Map
```

キーの型をIntに固定し、値の型を変えられる別名を作る。

```
type IntMap v = Map.Map Int v
```

次のように、最後の型引数を省略して定義する書き方もできる。

```
type IntMap = Map.Map Int
```

上の二つは代わりの書き方。どちらか一方を使う。

```
IntMap String  -- Map.Map Int Stringと同じ型
```

残った型引数は「値の型」。キーの型Intはすでに固定されている。

もう一つのIntMap

このIntMapは、自分で付けたMap.Map Intの別名。
Data.IntMapという、整数キー専用のモジュールもある。
そちらのIntMap型とは区別して読もう。

7.6 修飾付きインポート


```
import qualified Data.Map as Map
```

qualified：名前を修飾して使う。
Preludeのlookupやmapとの衝突を避けられる。

```
Map.lookup
Map.fromList
Map.map
```

as Map：Data.Mapというモジュールを、
短い名前Mapで参照する。

qualifiedとasは役割が異なる。
別名を付けるだけでは、非修飾の名前の衝突は防げない。

7.6 Map.Mapと型シノニム


```
import qualified Data.Map as Map

type IntMap = Map.Map Int
```

Map.Mapの前のMapは、インポートしたモジュールの別名。

後ろのMapは、
そのモジュールで定義されている型コンストラクタ。

```
Map.Map Int String
IntMap String
```

この二つは同じ型を表す。

型コンストラクタと値コンストラクタは別。
型シノニムの名前を使って、新しい値を作ることはできない。

7.6 Maybe：値がある／ないを表す

Maybe aは、a型の値がある場合と、値がない場合を表す。

```
-- 標準のMaybeの定義を読む（再定義しない）
data Maybe a = Nothing | Just a
```

```
Just "JAH3I" :: Maybe String
Nothing      :: Maybe String
```

ロッカー検索でも、成功したらJustで暗証番号を返し、
失敗したらNothingを返す設計にできる。

ただし、番号がない場合と使用中の場合をどちらもNothingにすると、
失敗理由を区別できない。

「値がない」だけでなく、
「なぜ失敗したか」も伝えたいときにEitherを使う。

7.6 Either型：Maybeとの違い


```
-- 標準のEitherの定義を読む（再定義しない）
data Either a b = Left a | Right b
    deriving (Eq, Ord, Read, Show)
```

Leftの中身の型はa、Rightの中身の型はb。
二つを同時に持つ組ではなく、どちらか一方の形を取る。

この使い方ではLeftに失敗理由、Rightに成功結果を入れる。

Either自体はエラー専用の型ではない。
二つの型のどちらか一方の値を持つデータ型。

7.6 Either型：Maybeとの違い

Maybeは値があるかないかを表す。
Eitherなら、失敗した理由も値として返せる。

Maybe Stringで表すと

```
Just "JAH3I"
Nothing
```

失敗したことは分かるが、
二つの理由を区別できない。

Either String Stringなら

```
Right "JAH3I"
Left "使用中"
Left "番号がない"
```

成功した暗証番号と、
失敗した理由を分けて持てる。

MaybeのNothingには、失敗の理由を格納する場所がない。
Eitherなら、Leftに理由を、Rightに成功結果を入れられる。

7.6 Either型：型引数を読む


```
ghci> :t Right 'a'
Right 'a' :: Either a Char
```

Rightの中身の型はCharに決まる。
Left側の型aは、まだ決まらない。

```
ghci> :t Left True
Left True :: Either Bool b
```

Leftの中身の型はBoolに決まる。
Right側の型bは、まだ決まらない。

値の形と、型宣言のどちらの型引数が対応するかを確認しよう。

問題の準備：型とロッカーのデータ


```
import qualified Data.Map as Map

data LockerState = Taken | Free
    deriving (Show, Eq)

type Code = String
type LockerMap = Map.Map Int (LockerState, Code)
```

LockerStateは使用中か空きかを表す。
各ロッカーの番号を、状態と暗証番号の組に対応させる。

問題の準備：型とロッカーのデータ


```
lockers :: LockerMap
lockers = Map.fromList
    [(100,(Taken,"ZD39I"))
    ,(101,(Free,"JAH3I"))
    ,(103,(Free,"IQSA9"))
    ,(105,(Free,"QOTSA"))
    ,(109,(Taken,"893JJ"))
    ,(110,(Taken,"99292"))]
```

100は使用中、101と105は空き。
102はデータに存在しない。

7.6 問題 Either型を使って失敗も型で表す

課題

番号からロッカーを検索する関数を実装しよう。

```
lockerLookup :: Int -> LockerMap
             -> Either String Code
```

① 番号が存在しなければ、理由をLeftの文字列で返す。

② 使用中の場合も、別の理由をLeftで返す。

③ 空いていれば、暗証番号をRightで返す。

7.6 問題 Either型を使って失敗も型で表す

穴埋めヒント

```
lockerLookup number lockerMap =
  case Map.lookup number lockerMap of
    __A__ ->
      Left ("Locker number " ++ show number
            ++ " doesn't exist!")
    Just (__B__, _) ->
      Left ("Locker " ++ show number
            ++ " is already taken!")
    Just (Free, code) -> __C__ code
```

Aは検索に失敗した場合のパターン、Bは使用中を表す状態。
Cには、成功結果を包む値コンストラクタを入れる。

7.6 問題 Either型を使って失敗も型で表す

解答例

```
lockerLookup number lockerMap =
  case Map.lookup number lockerMap of
    Nothing ->
      Left ("Locker number " ++ show number
            ++ " doesn't exist!")
    Just (Taken, _) ->
      Left ("Locker " ++ show number
            ++ " is already taken!")
    Just (Free, code) -> Right code
```

Map.lookupのNothingと、ロッカーが使用中の状態Takenは、
異なる失敗理由に対応する。

ロッカー検索：実装例と結果


```
ghci> lockerLookup 101 lockers
Right "JAH3I"

ghci> lockerLookup 100 lockers
Left "Locker 100 is already taken!"

ghci> lockerLookup 102 lockers
Left "Locker number 102 doesn't exist!"
```

成功・使用中・番号なしの三つの場合が、
戻り値から分かる。

7.7 再帰的なデータ構造

リストは、空のリストか、
先頭の要素と残りのリストを組にしたものとして表せる。

```
data List a = Empty
            | Cons a (List a)
            deriving (Show, Read, Eq, Ord)
```

Emptyは空のリストに対応する。
Consは一つの要素と、その後ろに続くリストを持つ。

定義の中にList a自身が現れるので、再帰的なデータ構造になる。

```
Empty                    -- []
Cons 5 Empty             -- [5]
Cons 4 (Cons 5 Empty)     -- [4,5]
```

7.7 再帰的なデータ構造

標準リストの(:)と[]に対応するものを、
自分の型ではConsとEmptyで表している。

```
[3,4,5]
3 : (4 : (5 : []))


Cons 3 (Cons 4 (Cons 5 Empty))
```

二つの書き方は、いずれも
「先頭の要素と、残りのリスト」という同じ構造を表す。

後ろのリストをさらに同じ形でたどれる。
末尾の空リストに到達すると、構造が終わる。

7.7 再帰的なデータ構造

値コンストラクタを演算子にすると、
標準リストの(:)に近い見た目で書ける。

```
infixr 5 :-:
data List a = Empty | a :-: (List a)
    deriving (Show, Read, Eq, Ord)
```

```
3 :-: 4 :-: 5 :-: Empty
-- 3 :-: (4 :-: (5 :-: Empty))
```

infixr 5は、右結合で、結合の強さが5という指定。
括弧を省いても、右からまとまる。

Cons版と演算子版は、代わりの定義。
同じモジュールに両方のListを定義しない。

infixr・infixlと結合方向

infixrは右結合、infixlは左結合を指定する。

```
infixr 5 :-:
3 :-: 4 :-: 5 :-: Empty
-- 3 :-: (4 :-: (5 :-: Empty))
```

```
-- 標準の(-)の設定を読む
infixl 6 -
10 - 3 - 2
-- (10 - 3) - 2 = 5
```

数字は結合の強さ（0〜9）。大きいほど強く結び付く。

r・lは、同じ優先度の式のまとまり方を指定する。
実行時の評価順序を指定するものではない。

中置演算子の優先度（9〜5）

優先度	演算子	結合方向	用途
9	`.`	右	関数合成
9	`!!`	左	リストの要素取得
8	`^`, `^^`, `**`	右	累乗
7	`*`, `/`, `` `div` ``, `` `mod` ``	左	掛け算・割り算・余り
6	`+`, `-`	左	足し算・引き算
5	`:`, `++`	右	リストの構築・連結

数字が大きいほど優先。累乗、掛け算、足し算の順にまとまる。

```
2 + 3 * 4 ^ 2
-- 2 + (3 * (4 ^ 2)) = 50
```

自作の:-:も、標準の:と同じinfixr 5にしている。

中置演算子の優先度（4〜0）

優先度	演算子	結合方向	用途
4	==, /=, <, <=, >, >=	非結合	比較
3	&&	右	論理AND
2	||	右	論理OR
1	>>, >>=	左	処理の連結
0	$	右	関数適用

非結合（infix）の比較演算子では、a < b < cとは書けない。
複数の比較は、それぞれの結果を&&などで組み合わせる。

```
1 < 2 && 2 < 3
-- (1 < 2) && (2 < 3)
-- True
```

比較の優先度4は、&&の3より高い。

7.7.1 自作リストの結合関数（＋＋＋）

左のリストが空なら、右のリストを返す。
空でなければ、先頭を残して、残りと右のリストを結合する。

標準リストの結合

```
infixr 5 ++
(++) :: [a] -> [a] -> [a]
[] ++ ys = ys
(x:xs) ++ ys = x : (xs ++ ys)
```

自作リストの結合

```
infixr 5 +++
(+++) :: List a -> List a -> List a
Empty +++ ys = ys
(x :-: xs) +++ ys = x :-: (xs +++ ys)
```

7.7.1 自作リストの結合関数（＋＋＋）

型やコンストラクタの名前が変わっても、
再帰の進め方は標準リストと同じ。

```
ghci> [1,2,3] ++ [4,5]
[1,2,3,4,5]
```

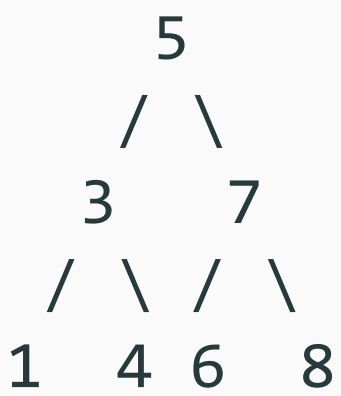
```
ghci> let a = 1 :-: 2 :-: 3 :-: Empty
ghci> let b = 4 :-: 5 :-: Empty
ghci> a +++ b
1 :-: (2 :-: (3 :-: (4 :-: (5 :-: Empty))))
```

対応：[]はEmpty、(:)は(:-:)、(++)は(+++)。
左のリストを順にたどり、右のリストにつなげる。

7.7 再帰的なデータ構造：二分探索木

各節は値と、
左右二つの部分木を持つ。

根より左の部分木の
全要素は小さく、
右の部分木の全要素は大きい。



親と子だけを比較する条件では
不十分。部分木の全体について
成り立つ必要がある。

Treeという型の定義だけでは、この大小関係は保証されない。
挿入処理で条件を保つ。

今回の実装では、同じ値がすでにあれば木をそのまま返す。

二分探索木：型の定義と補助関数


```
data Tree a = EmptyTree
            | Node a (Tree a) (Tree a)
            deriving Show
```

EmptyTree：空の木。Node：値、左部分木、右部分木を持つ。

```
singleton :: a -> Tree a
singleton x = Node x EmptyTree EmptyTree
```

singletonは、左右が空で、値を一つだけ持つ木を作る。

挿入先が空の木に到達したとき、この形を使う。

問題 二分探索木を実装しよう

課題

要素を一つ挿入した、新しい二分探索木を返す関数を実装しよう。

```
treeInsert :: Ord a
           => a -> Tree a -> Tree a
```

二分探索木の大小関係を保つこと。
同じ値がすでにある場合は、木をそのまま返す。

問題 二分探索木を実装しよう

穴埋めヒント

```
treeInsert :: Ord a
  => a -> Tree a -> Tree a
treeInsert x __A__ =
    singleton x
treeInsert x (Node a l r)
  | x == a = Node a l r
  | x < a =
      Node a (__B__) r
  | x > a =
      Node a l (__C__)
```

Aは挿入先が空の場合のパターン。
B・Cには、進む側の部分木を更新する再帰呼び出しを入れる。

問題 二分探索木を実装しよう

解答例

```
treeInsert :: Ord a
  => a -> Tree a -> Tree a
treeInsert x EmptyTree =
    singleton x
treeInsert x (Node a l r)
  | x == a = Node a l r
  | x < a =
      Node a (treeInsert x l) r
  | x > a =
      Node a l (treeInsert x r)
```

挿入先が空なら、値を一つだけ持つ木を作る。
値が等しければそのまま、小さければ左、大きければ右へ進む。

既存の木を書き換えるのではなく、挿入後の新しい木を返す。

問題 二分探索木を実装しよう

課題

指定した値が木に含まれるかを調べる関数を実装しよう。

```
treeElem :: Ord a
         => a -> Tree a -> Bool
```

含まれる場合はTrue、含まれない場合はFalseを返す。

問題 二分探索木を実装しよう

穴埋めヒント

```
treeElem :: Ord a
  => a -> Tree a -> Bool
treeElem _ EmptyTree = __A__
treeElem x (Node a l r)
  | x == a = __B__
  | x < a = __C__
  | x > a = __D__
```

A・Bには探索結果を表すBoolの値を入れる。
C・Dには、比較結果に応じた側を探索する式を入れる。

問題 二分探索木を実装しよう

解答例

```
treeElem :: Ord a
  => a -> Tree a -> Bool
treeElem _ EmptyTree = False
treeElem x (Node a l r)
  | x == a = True
  | x < a = treeElem x l
  | x > a = treeElem x r
```

探索でも、比較結果に応じて片方の部分木だけを調べる。
空の木まで到達したら、その値は存在しない。

Ord aは、要素の大小を比較するための型クラス制約。

二分探索木：実行結果

foldrでリストを右からたたみ込み、
空の木に対してtreeInsertを繰り返す。

```
ghci> let nums = [8,6,4,1,7,3,5]
ghci> let numsTree =
        foldr treeInsert EmptyTree nums
```

```
Node 5
  (Node 3 (Node 1 EmptyTree EmptyTree)
          (Node 4 EmptyTree EmptyTree))
  (Node 7 (Node 6 EmptyTree EmptyTree)
          (Node 8 EmptyTree EmptyTree))
```

初期値はEmptyTree。最初に挿入する5が根になる。

二分探索木：実行結果


```
ghci> 8 `treeElem` numsTree
True
ghci> 100 `treeElem` numsTree
False
ghci> 1 `treeElem` numsTree
True
ghci> 10 `treeElem` numsTree
False
```

木に含まれる値ならTrue、含まれなければFalse。
比較によって、探索する側を絞り込んでいる。

型クラス中級講座

型クラスは、ある型に対して使える共通の操作を定義する。

Eqなら等値比較、Ordなら大小比較、Showなら文字列への変換。

型Tが型クラスCのインスタンスであるとは、
Cで定めた操作を、T型の値に対して使えるということ。

```
data TrafficLight = Red | Yellow | Green
```

data：値の形を定義する。

class：共通の操作を定義する。

instance：その型での操作の実装を与える。

PythonやJavaでオブジェクトを作るためのclassとは、役割が異なる。

これまではderivingに任せていた実装を、この節では手書きし、
操作の振る舞いを自分で決める。

7.8 型クラス中級講座

classとinstanceの役割を比べる

class

```
class Eq a where
  (==) :: a -> a -> Bool
```

標準Eqの定義の抜粋。
共通の操作と、その型を定める。

aは、インスタンスにする型を
当てはめる場所。

instance

```
instance Eq TrafficLight
  where
    Red == Red = True
    Yellow == Yellow = True
    Green == Green = True
    _ == _ = False
```

特定の型での処理を実装する。

左のaにTrafficLightを当てはめ、
信号機どうしの(==)を右のように実装する。

classには、メソッドの型宣言や既定の実装を書ける。
instanceには、その型でのメソッドの実装を書く。

7.8 Eq型クラスの内部


```
-- 標準Eqの説明用の抜粋
class Eq a where
  (==), (/=) :: a -> a -> Bool
  x == y = not (x /= y)
  x /= y = not (x == y)
```

最小完全定義：(==)か(/=)の少なくとも一方を、自分で実装する。

片方を実装すれば、もう片方の既定の実装が使える。

両方とも省略すると、
既定の実装がお互いを呼び合い、比較が終わらない。

7.8 Eq型クラスの内部

instance宣言の書き方

基本形：instance 型クラス名 対象の型 where

```
instance Eq TrafficLight where
  Red    == Red    = True
  Yellow == Yellow = True
  Green  == Green  = True
  _      == _      = False
```

Eqは型クラス名、TrafficLightは対象となる型。
where以下に、その型でのメソッドの実装を書く。

この例では(==)を定義する。
(/=)には、Eqで用意された既定の実装を使える。

同じ色どうしをTrueとし、残りの組合せを_でFalseにする。
すべての組を一つずつ列挙せずに済む。

7.8 EqとShowの動作を確かめる

Eqのインスタンスを作ると、比較やelemを使える。

```
ghci> Red == Red
True
ghci> Red == Yellow
False
ghci> Red `elem` [Red,Yellow,Green]
True
```

この等値比較はderiving Eqでも同じ結果になる。
ここでは、仕組みを理解するために手書きしている。

7.8 EqとShowの動作を確かめる

Showの表示を自分で決めたい場合は、showを実装する。

```
instance Show TrafficLight where
  show Red = "Red light"
  show Yellow = "Yellow light"
  show Green = "Green light"

ghci> Red
Red light
```

deriving ShowならRedと表示する。
掲載の手動実装ならRed lightと表示する。表示結果は同じではない。

GHCiの表示にもShow

GHCiに値を入力すると、通常はprintで結果が表示される。
printはShowのshowを使って、値を文字列に変換する。
Red lightと出るのも、自作したShowが使われるため。

7.8 サブクラス化

ある型クラスに属するために、
別の型クラスに属することを要求できる。

```
-- 教科書p.145の説明で使われる古い定義の形
class Eq a => Num a where ...
```

現在の確認環境では、NumにEqの制約はない。

```
-- GHC 9.10.3で確認できる前提関係
class Eq a => Ord a where ...
```

Ordのインスタンスにする型は、
Eqのインスタンスでもある必要がある。

クラス宣言の制約は、型クラス同士の前提関係を表す。
次のMaybeの例では、インスタンス宣言の制約として
中身に必要な操作を要求する。

7.8 Maybeの比較には何が必要か

Justどうしの比較では、中身の値を比較する。

```
Just x == Just y = x == y
```

Maybe Int、Maybe Charなどを別々に定義せず、
Maybe aの形でまとめて扱う。
ただし、中身を比較する条件Eq aは必要。

xとyの型はa。
右辺で(==)を使うには、中身を比較できるEq aが必要。

Maybeだけでは、値が持てる型にならない。
Eqの引数には、型引数を与えたMaybe aを使う。

7.8 Maybeの比較には何が必要か


```
-- 既存のMaybeのインスタンスを読む（再定義しない）
instance Eq a => Eq (Maybe a) where
    Just x  == Just y  = x == y
    Nothing == Nothing = True
    _       == _       = False
```

Eq aは要求する条件、Eq (Maybe a)は提供する比較の操作。

Eqが受け取るのは、Maybeという型コンストラクタではなく、Maybe aという値の型。

7.8 GHCiで型と型クラスを調べる

型について調べるには、:infoの後ろに型名を書く。

```
ghci> :info Maybe
data Maybe a = Nothing | Just a
instance Eq a => Eq (Maybe a)
instance Ord a => Ord (Maybe a)
instance Functor Maybe
-- 出力の一部。他のインスタンスも表示される
```

型クラスについて調べるには、その型クラス名を書く。

```
ghci> :info Eq
-- Eqが提供する操作とインスタンスなどを確認

ghci> :info Num
-- Numについての情報を確認
```

表示内容はGHCやライブラリの版により異なる。

7.9 YesとNoの型クラス

JavaScriptでは、数値や文字列も条件として解釈される。
Haskellでは、条件にはBoolを使う。

この節では、いろいろな型の値を
「yesっぽい」「noっぽい」と判定する共通の操作を、
自分で作ってみる。

0や空のリストをFalseにするなど、
何をyesと見なすかは、型ごとのインスタンスで決める。

```
class YesNo a where
    yesno :: a -> Bool
```

値を受け取る関数なので、
aにはMaybe IntやTree Intのような、値が持てる型を指定する。

7.9 YesとNoの型クラス

JavaScriptでは、0・空文字列・falseは偽として扱われる。

```
if (0) alert("YEAH!");
else alert("NO!");

if ("") alert("YEAH!");
else alert("NO!");

if (false) alert("YEAH!");
else alert("NO!");
```

空でない文字列なら、真として扱われる。

```
if ("WHAT") alert("YEAH!");
else alert("NO!");
```

7.9 YesNo型クラスの定義

Haskellのifの条件は、常にBoolでなければならない。

```
-- そのままでは型が合わない
if 1 then "YES" else "NO"
```

```
class YesNo a where
    yesno :: a -> Bool
```

yesnoで、その型の値をBoolへ変換する規則を定義する。

```
if yesno (1 :: Int) then "YES" else "NO"
-- "YES"
```

YesNoのインスタンスにしても、
ifが数値を直接受け取るようになるわけではない。

7.9 型ごとのYesNoの規則


```
instance YesNo Int where
  yesno 0 = False
  yesno _ = True

instance YesNo Bool where
  yesno = id

instance YesNo [a] where
  yesno [] = False
  yesno _  = True
```

Intでは0かどうか、リストでは空かどうかを判定する。

Boolのidは、受け取った値をそのまま返す関数。
TrueとFalseを、変換せずに判定結果として使っている。

7.9 型ごとのYesNoの規則


```
instance YesNo (Maybe a) where
  yesno (Just _) = True
  yesno Nothing  = False

yesno (Just False)  -- True
yesno (Just 0)      -- True
yesno Nothing       -- False
```

Maybeでは、中身ではなくJustかNothingかだけを見る。
だから中身のaに対する型クラス制約は不要。

Intでは0かどうか、リストでは空かどうかを判定する。

中身を比較しないので、aの型が何であっても同じ規則を使える。

7.9 YesとNoの型クラス

木と信号機にも、型ごとの判定規則を与えられる。

```
instance YesNo (Tree a) where
    yesno EmptyTree = False
    yesno _ = True
```

空の木はFalse、空でない木はTrueとする。

```
instance YesNo TrafficLight where
    yesno Red = False
    yesno _ = True
```

この例の信号機では、赤をFalse、黄・青をTrueとする。

7.9 YesとNoの型クラス

同じyesnoでも、引数の型に応じた規則で判定する。

```
ghci> yesno $ length []
False
ghci> yesno "haha"
True
ghci> yesno ""
False
ghci> yesno $ Just 0
True
```

```
ghci> yesno True
True
ghci> yesno EmptyTree
False
ghci> yesno []
False
ghci> yesno [0,0,0]
True
```

```
ghci> :t yesno
yesno :: YesNo a => a -> Bool
```

7.9 YesNo値を取る関数


```
yesnoIf :: YesNo y => y -> a -> a -> a
```

第1引数は判定する値。yesnoでBoolに変換して判定する。

第2引数はyesのときの結果、第3引数はnoのときの結果。

二つの結果は、同じa型でなければならない。

```
yesnoIf (Just 500) "YEAH!" "NO!"
-- yesno (Just 500)はTrueなので "YEAH!"

yesnoIf Nothing "YEAH!" "NO!"
-- yesno NothingはFalseなので "NO!"
```

Haskellでは、選ばれなかった側の結果は、
この分岐のためには評価されない。
教科書は、この簡潔な実装と遅延評価の関係にも触れている。

7.9 YesNo値を取る関数


```
yesnoIf :: YesNo y => y -> a -> a -> a
yesnoIf yesnoVal yesResult noResult =
    if yesno yesnoVal
        then yesResult
        else noResult
```

yesnoValは判定する値。
yesResultとnoResultは、判定に応じて選ぶ結果。

```
yesnoIf [] "YEAH!" "NO!"        -- "NO!"
yesnoIf [2,3,4] "YEAH!" "NO!"   -- "YEAH!"
yesnoIf True "YEAH!" "NO!"      -- "YEAH!"
```

選ばれない側の計算

yesnoIf True "YES" (error "unused")は"YES"になる。
Haskellは遅延評価を使うので、この例では
選ばれなかった側のerrorを評価せずに結果を返せる。

7.10 Functor型クラス

リストのmapは、リストの各要素に関数を適用する操作だった。
Functorは、こうした「中身を写す」操作を共通に扱う。

教科書では、リストやMaybeを、
値が入っている箱にたとえている。
空の場合もあれば、値が入っている場合もある。

```
fmap (*2) [1,2,3]   -- [2,4,6]
fmap (*2) (Just 3)  -- Just 6
fmap (*2) Nothing   -- Nothing
```

同じ関数を使っても、中身への適用の仕方は型によって異なる。
その違いをFunctorのインスタンスに実装する。

この節では、リスト・Maybe・Tree・Either・Mapを順に見て、
fmapに共通する型を読み解いていく。

7.10 Functor型クラス


```
-- 標準Functorの説明用の抜粋（再定義しない）
class Functor f where
    fmap :: (a -> b) -> f a -> f b
```

fmapは、a型の値をb型へ変換する関数と、f a型の値を受け取る。

返す値の型はf b。fは同じままで、要素の型aはbに変わってよい。

```
Eq a        -- aには値の型が入る
Functor f   -- fには型コンストラクタが入る
```

例：Eq (Maybe Int)に対し、Functor Maybe。

Eqのaは値の型だったが、Functorのfは型を一つ受け取るもの。
fをMaybeに置き換えると、入力がMaybe a、出力がMaybe bになる。

7.10 Functor型クラス

mapはリストの各要素を変換する。
fmapでは、その考え方をリスト以外にも広げている。

```
map  :: (a -> b) -> [a] -> [b]
fmap :: (a -> b) -> f a -> f b
```

リストの場合、fに対応するのは[]という型コンストラクタ。
型引数を一つ与えると、[a]や[b]のような値の型になる。

型宣言のfを[]に置き換えると、
fmapの型はリスト用のmapと同じ形になる。

---

fmapは記号でも書ける

(<$>)はfmapの中置演算子版。
length <$> Just "cat"は、fmap length (Just "cat")と同じ。
どちらもJust 3になる。

7.10 Functor型クラス


```
-- 既存のインスタンスを読む（再定義しない）
instance Functor [] where
    fmap = map
```

指定するのは[a]ではなく、型を一つ受け取る[]。
これは空リストという値ではなく、リスト型の型コンストラクタ。

```
ghci> fmap (*2) [1..3]
[2,4,6]
ghci> map (*2) [1..3]
[2,4,6]
```

リストに使った結果は一致する。
空リストに使えば、どちらも空リストを返す。

7.10.1 MaybeはFunctorだよ、たぶん


```
-- 既存のインスタンスを読む（再定義しない）
instance Functor Maybe where
    fmap f (Just x) = Just (f x)
    fmap _ Nothing  = Nothing
```

指定するのはMaybe。(Maybe a)ではなく、
型引数を一つ取る型コンストラクタ。

```
-- fをMaybeに置き換えたfmapの型
(a -> b) -> Maybe a -> Maybe b
```

```
fmap (*2) (Just 200)  -- Just 400
fmap (*2) Nothing     -- Nothing
```

Justなら、中身を取り出して関数を適用し、
結果をもう一度Justで包む。
Nothingは空の箱に相当し、写す中身がないのでそのまま返す。

7.10 fmapでは要素の型も変えられる


```
fmap :: (a -> b) -> f a -> f b
```

```
fmap show (Just (3 :: Int))
-- Just "3" :: Maybe String
```

f = Maybe、a = Int、b = String。
外側のMaybeは同じで、中身の型が変わる。

```
-- この例に合わせたfmapの型
(Int -> String) -> Maybe Int -> Maybe String
```

```
fmap show (Nothing :: Maybe Int)
-- Nothing :: Maybe String
```

値がない場合でも、入力と出力で型が変わることはある。

7.10.2 TreeもFunctorの森に


```
instance Functor Tree where
    fmap _ EmptyTree = EmptyTree
    fmap f (Node x left right) =
        Node (f x) (fmap f left) (fmap f right)
```

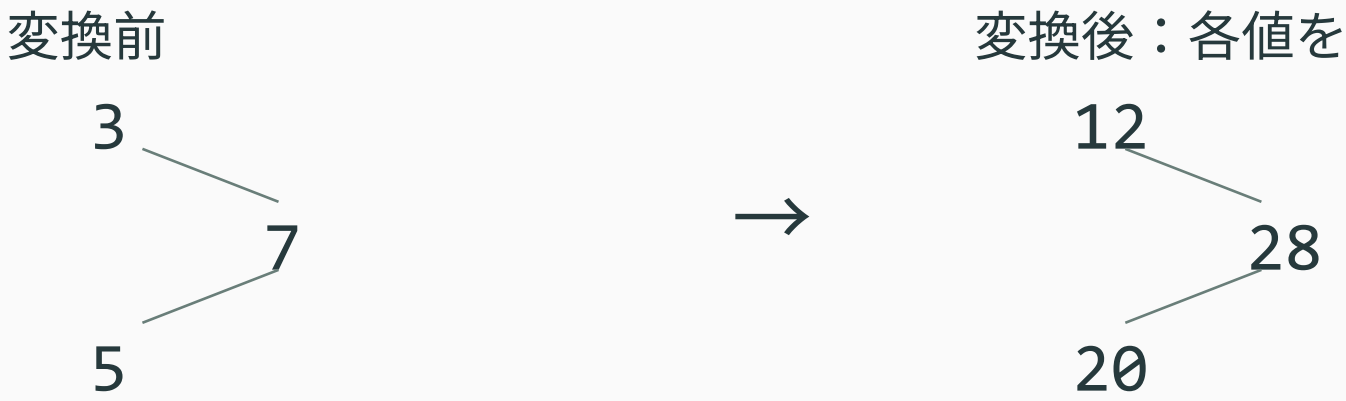
空の木は空のまま。
Nodeなら、その値に関数を適用し、左右の部分木も再帰的に写す。

```
(a -> b) -> Tree a -> Tree b
```

木が持つ値を、それぞれ同じ関数で変換する。
枝の形は保たれる。

7.10.2 TreeもFunctorの森に


```
fmap (*4) (foldr treeInsert EmptyTree [5,7,3])
```



枝の形は同じで、各値が4倍になる。
ただし、negateで写すと二分探索木の大小関係は崩れる。

この式の根は12。教科書の掲載結果（根20）とは異なるため、
ここでは実際の計算結果に合わせている。

7.10.3 EitherはFunctorであるか否か


```
-- 既存のインスタンスを読む（再定義しない）
instance Functor (Either e) where
    fmap f (Right x) = Right (f x)
    fmap _ (Left e)  = Left e
```

Eitherは型引数を二つ取る。
第1引数を固定したEither eをFunctorにする。

```
fmap :: (a -> b)
     -> Either e a -> Either e b
```

型引数を一つ残したEither eが、Functorのfに対応する。

7.10.3 EitherはFunctorであるか否か


```
fmap length (Right "cat" :: Either String String)
-- Right 3
fmap length (Left "error" :: Either String String)
-- Left "error"
```

関数が受け取るのはRightの中身の型a。
Leftの中身の型eはaと同じとは限らないので、
同じ関数を適用できず、そのまま残す。

第1の型引数eは固定したまま。
変えられるのは、第2の型引数とRight側の値。

同じ関数で両側を写そうとしても、
Leftの中身とRightの中身は、そもそも違う型かもしれない。

7.10 Mapの値を写す


```
import qualified Data.Map as Map

m = Map.fromList [(1,"cat"),(2,"haskell")]
    :: Map.Map Int String
```

```
fmap length m
-- fromList [(1,3),(2,7)]
```

Map k vという値について、キーkを保ち、値vに関数を適用する。

```
(a -> b) -> Map.Map k a -> Map.Map k b
```

Mapも値の入った箱と考えられる。
fmapではキーとの対応を保ち、それぞれの値だけを新しい値へ写す。

7.10 Mapの値を写す

課題

MapのFunctorインスタンスの形を考えよう。
fに対応する型コンストラクタも考える。

```
fmap :: (a -> b) -> f a -> f b
```

値だけを変換するMap.mapの型は次のとおり。

```
Map.map :: (a -> b)
        -> Map.Map k a -> Map.Map k b
```

キーを保ったまま、値に関数を適用する。

標準ライブラリの既存インスタンスを考える課題。
同じインスタンスを再定義せず、実装の形を紙に書いてみよう。

7.10 Mapの値を写す

穴埋めヒント

```
-- 既存の実装の形を読む。再定義しない
instance Functor (__A__) where
    fmap = __B__
```

Aには、キーの型を固定した型コンストラクタを入れる。
Bには、値だけを写す既存の関数を入れる。

7.10 MapとFunctor：解答と実装の形

解答例

fに対応するのはMap.Map k。
キーの型を固定し、値の型を一つ受け取る。

```
-- 既存の実装の形を読む。再定義しない
instance Functor (Map.Map k) where
    fmap = Map.map
```

```
Map.map :: (a -> b)
        -> Map.Map k a -> Map.Map k b
```

ここでのMap.mapは、リスト用のPrelude.mapとは異なる関数。

値だけを変換し、キーの比較を行わないので、
この操作にOrd kの制約は不要。

この説明は確認環境GHC 9.10.3／containers 0.7に合わせている。

7.11 型を司るもの、種類

値には型が付く。型にも、その型がどんな引数を取るかを表す「種類（kind）」が付く。

Maybeは型を一つ受け取り、Maybe Intのような型を作る。
この働きは、関数が引数を受け取って結果を返す姿に似ている。

```
Maybe                  -- 型を一つ待っている
Maybe Int              -- 値が持てる型

Either String          -- あと一つ、型を待っている
Either String Int      -- 値が持てる型
```

型コンストラクタにも部分適用ができる。
この違いを記号で表すと、
なぜFunctorにMaybeやEither Stringを渡すのかが分かる。

次のページでは、GHCiの:kを使って種類を確認する。

7.11 型を司るもの、種類

型は値に付くラベル。種類（kind）は型に付くラベル。

```
ghci> :t (3 :: Int)
(3 :: Int) :: Int
ghci> :k Int
Int :: *
```

:tは値の型を調べる。:kは型の種類を調べる。

```
ghci> :k Maybe
Maybe :: * -> *
ghci> :k Maybe Int
Maybe Int :: *
```

*は、この章では値の型を表す。
Maybeは一つの型を受け取り、Maybe Intのような値の型を作る。

7.11 型引数と部分適用


```
Either             :: * -> * -> *
Either String      :: * -> *
Either String Int  :: *
```

型引数を与えると、残る引数の数が減る。

```
Eq (Either String Int)
-- Eqが受け取る型のkindは *

Functor (Either String)
-- Functorが受け取る型のkindは * -> *
```

Functorのfは型コンストラクタ。f aとf bは値の型になる。

Functorのfには、型を一つ受け取って値の型を作るものが必要。
Eitherは二つ取るため、一つ与えたEither Stringなどを指定する。

第8回のまとめ

7.6 型シノニム：既存の型に、意味のある別名を付ける。

7.7 再帰的なデータ構造：値の形に沿って関数も再帰する。

7.8 型クラス中級講座：class・instance・制約を使い分ける。

7.9 YesNo：型ごとの規則で、値をBoolに変換する。

7.10 Functor：型コンストラクタごとにfmapを実装する。

7.11 種類（kind）：EqとFunctorの引数の形の違いを説明する。